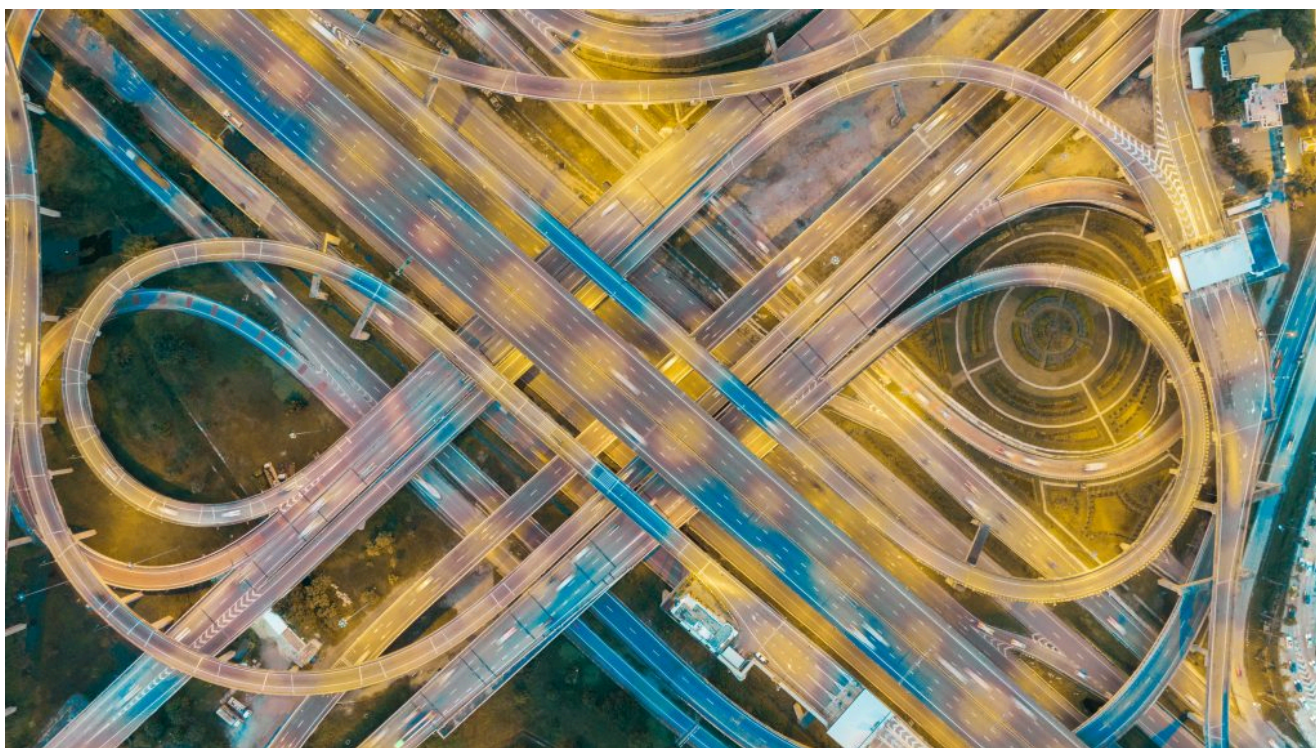


АГЕНТЫ ИИ / ИНФРАСТРУКТУРА ИИ / ОБЛАЧНЫЕ СЕРВИСЫ

AWS, Microsoft и Google сходятся во мнении, что сессия — это новая единица вычислений. Но они расходятся во взглядах на то, как ее изолировать.

В AWS, Microsoft, Google и Anthropic среды выполнения агентов перестраиваются с учетом сеанса, а не запроса, и выполнение с учетом сеанса быстро становится стандартом для ИИ промышленного уровня.

26 июня 2026 года, 10:11 [Джанакiram MSV](#)



Getty Images для Unsplash+

За последние несколько месяцев четыре гиганта в сфере искусственного интеллекта незаметно перестроили одну и ту же систему: AWS, Microsoft, Google и Anthropic выпустили обновления среды выполнения агентов, которые указывают на один и тот же архитектурный сдвиг.

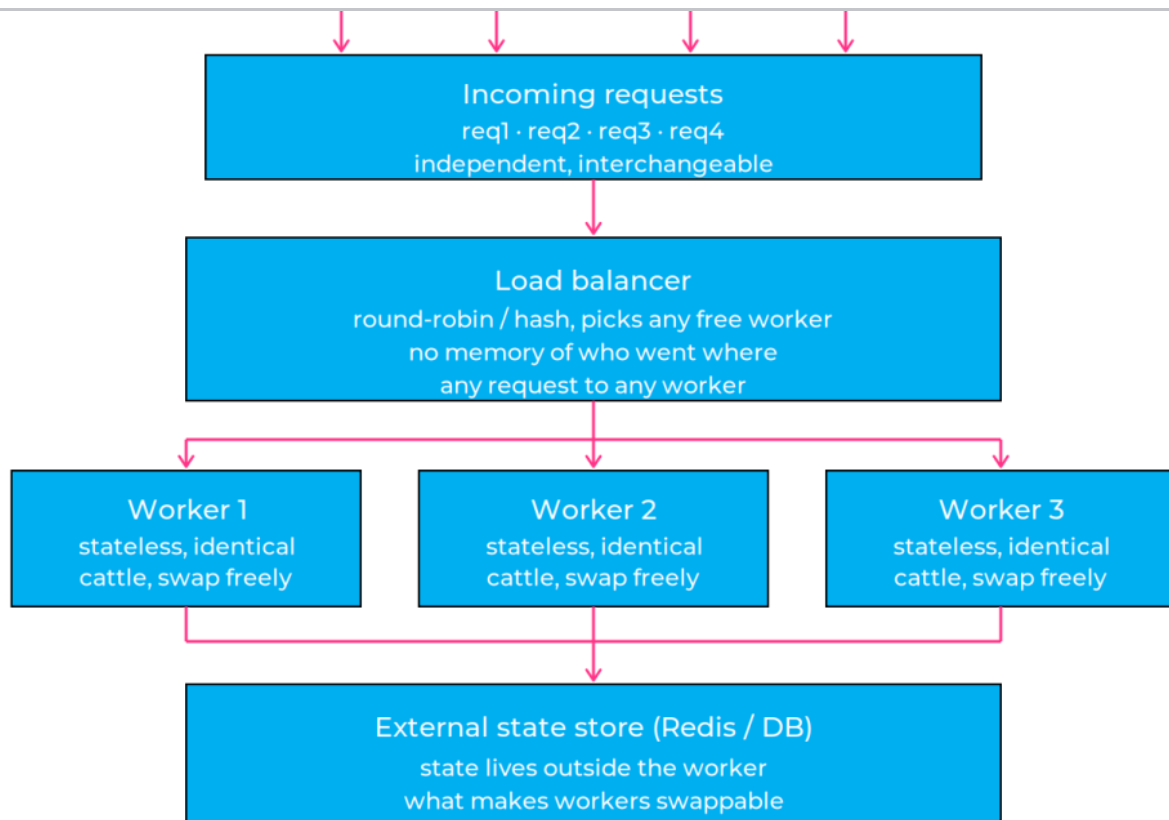
В апреле компания Microsoft перестроила свои агенты, размещенные на платформе Foundry, с учетом изоляции по сеансам. **Управляемые агенты**

микровиртуальную машину, а Google изолирует выполнение кода агента в специальной «песочнице». Этот общий подход — не просто еще одна функция модели или инструмент для разработчиков. Это переход от балансировки нагрузки на уровне запросов к выполнению с учетом сеанса.

Это важно, потому что корпоративные агенты — это не обычные вызовы API. Это длительные процессы с отслеживанием состояния, использующие инструменты, которые часто запускают код, зависящий от пользовательского ввода. По сути, среда выполнения агента становится плоскостью управления состоянием, идентификацией, изоляцией и жизненным циклом, а единица планирования, которую она использует, перешла от отдельного запроса к сеансу работы агента. Сближение четырех платформ говорит о том, что выполнение с учетом сеанса работы становится базовым требованием для агентов промышленного уровня.

Традиционная модель облачного масштабирования

Чтобы понять, почему это важно, стоит начать с того, что традиционная модель делала правильно. **Уровни NGINX** и HAProxy обычно располагаются перед пулом рабочих серверов и направляют каждый входящий запрос на следующий доступный бэкенд. Состояние намеренно хранится в другом месте, в Redis или базе данных, поэтому любой рабочий сервер может обработать любой запрос. Именно такая внешняя локализация состояния обеспечивает эластичное масштабирование, отказоустойчивость и замену инфраструктуры без ущерба для работы приложения. «Липкие» сессии были доступны, но компании использовали их как исключение для приложений с отслеживанием состояния, а не как стандарт.



The model rests on two assumptions: that requests do not depend on one another and that any backend can serve any request. For web APIs and microservices, those assumptions held for nearly two decades and enabled the scaling of the largest systems ever built. Agents challenge both at once, which is why conventional load balancing alone is no longer sufficient.

Why agents break the model

Agents pose two distinct challenges: preserving conversational state across turns and enforcing a security boundary strong enough to withstand untrusted code. Only the first can be addressed by routing alone.

TRENDING STORIES

1. [GitLab just surveyed 1,500 developers. Here's why it matters for your codebase.](#)
2. [AWS, Microsoft, and Google agree the session is the new unit of compute. They disagree on how to isolate it.](#)
3. [Neoclouds, sovereign AI and Postgres: The new operating model for regulated enterprises](#)

5. "A data lake of nuance for AI agents to swim in": AWS Context gets shipshape on reasoning

Why can the agent state not be pooled?

Imagine an enterprise support agent processing a refund. It reads the order, calls a tool, and then waits for the model. The next turn asks a clarifying question. If that turn is routed to a different replica without access to the prior context, the agent loses the state it needs to finish the workflow.

The load balancer starts to behave less like a traffic distributor and more like a session router.

The industry already has a name for the workaround, session affinity, and it already concedes the cost. Load-balancing approaches increasingly rely on consistent hashing, using a conversation identifier to keep related steps in a warm state. In plain English, the load balancer starts to behave less like a traffic distributor and more like a session router.

Why the trust boundary forces isolation

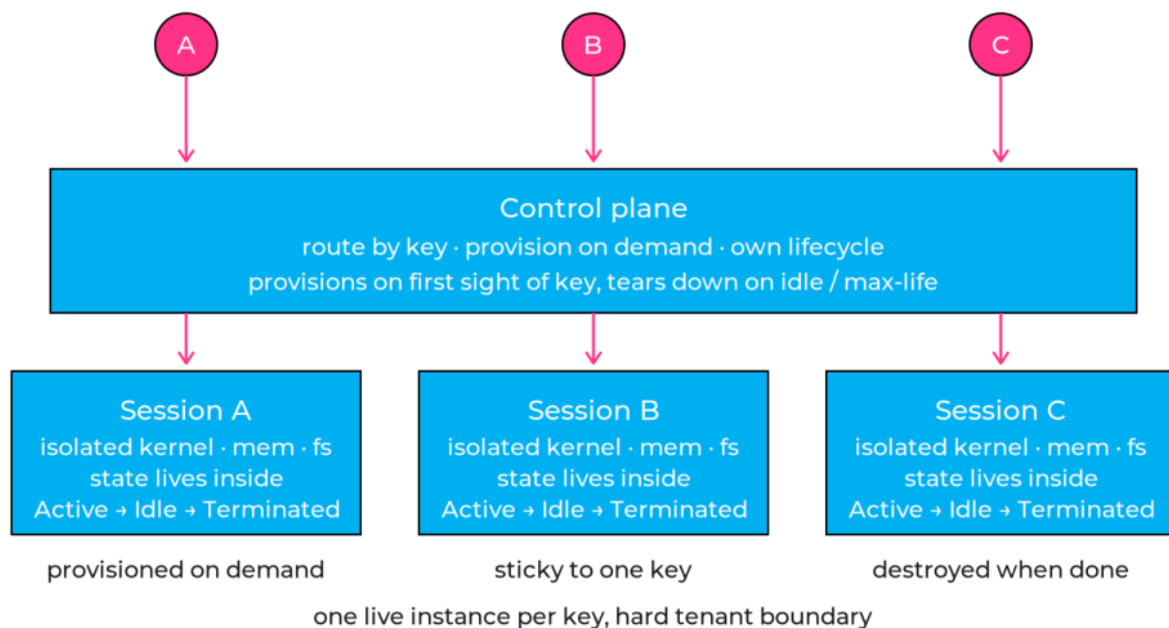
The second challenge is harder because it turns on isolation rather than on the state alone. An agent can execute model-generated code shaped by user input, so the backend must be treated as a security boundary rather than a generic compute target. A shared kernel does not give untrusted, agent-generated code the tenant isolation that enterprise security teams require. Session affinity can preserve routing continuity, but it does not by itself provide per-tenant isolation for untrusted execution. For platform and security teams, that is the line where a tuned load balancer is no longer enough.

This is not a theoretical concern. When **Asana** disclosed a flaw in its MCP server in June 2025, the server had been live since May 1, a window of about five weeks. A tenant-isolation check failed, and around 1,000 organizations could see project data belonging to other customers. Based on the reported details, security researchers traced the issue to a server that validated the user but did not consistently enforce the agent and tenant context behind cached responses. No

consistently bound together.

The session as a unit of compute

Once that binding becomes a correctness and security requirement, the session becomes the unit that the platform schedules. The clearest evidence is in the lifecycle. A session now has a state it never had before. AWS documents that an AgentCore session remains Active while it processes work and sits Idle when it is provisioned but waiting. It reaches Terminated after a 15-minute idle timeout or an 8-hour maximum lifetime. That is the lifecycle of a long-running execution environment, not of a short-lived HTTP request.



The economics follow the same logic. When billing is tied to active sessions, concurrency, idle time, and agent sizing become the cost drivers rather than request volume. That puts a new line item in front of platform teams and finance. The better mental model is no longer a traditional load balancer. It is closer to a virtual actor runtime, where an addressable identity is instantiated on demand, kept active while it is needed, and deactivated when idle, with one live instance per key.

Four approaches to the same shift

All four platforms are moving away from treating stateful, untrusted agent work as ordinary load-balanced traffic. The key difference is the compute primitive each provider chooses for the isolated execution environment.

3 AgentCore is the most opinionated of the four. Each session gets a dedicated **firecracker** microVM with isolated compute, memory, and filesystem, and a request

Sanitized.

The agents hosted on Microsoft Foundry implement a similar model through a different set of primitives. The platform creates a per-session VM-isolated sandbox on demand, runs it, and tears it down at session end, with no replica count and no warm pool to size. Each agent gets a dedicated Microsoft Entra identity, and sessions run a 15-minute idle timeout against a 30-day maximum lifetime.

Google **Agent Engine** is the most instructive hybrid. Its runtime keeps request scaling within the reasoning loop, a managed environment with configurable minimum and maximum instance counts, and a `container_concurrency` that defaults to 9. Even where Google retains request scaling for the loop, it separates untrusted code execution into an isolated Code Execution sandbox and externalizes conversation state to Sessions and Memory Bank. The provider that kept the load balancer in the loop still would not point it at stateful untrusted work.

Anthropic provides the clearest decomposition of the architecture. Managed Agents virtualize the agent into three components: a session that logs everything that happens, a harness that runs the loop and routes tool calls, and a sandbox where code runs. The harness becomes a near-stateless control plane, while the sandbox becomes a callable, rebuildable resource. The **Cloudflare** integration shows that the substrate can be decoupled. The agent loop runs on Anthropic while each tool call runs in a Cloudflare sandbox, which can be a full microVM or a lighter V8 isolate.

The four platforms converge on the routing and lifecycle model and diverge on the execution substrate, as the table shows.

Platform	Compute primitive per session	What it means for an enterprise buyer
AWS AgentCore	Dedicated Firecracker microVM, routed by session ID, 8-hour ceiling	The most opinionated answer, with isolation and routing fused at the microVM
Azure Foundry hosted agents	Per-session VM-isolated sandbox, dedicated Entra identity, 30-day max	Identity-led isolation suited to longer multi-day workloads
Google Agent Engine	Request-scaled loop plus a separate isolated code-execution sandbox	A hybrid that load-balances the reasoning and isolates the execution

Anthropic Managed Agents	Decoupled harness and sandbox, microVM or V8 isolate behind the loop	The most portable, with the substrate treated as a swappable layer
--------------------------	--	--

No single architecture is the right answer for every enterprise workload. A multi-day research agent fits Azure's longer-lived session, a code-heavy agent needs microVM-grade isolation, and high-volume automation favors a lighter isolate. Many enterprise platforms will end up combining more than one of these patterns.

The binding the application still owns

The platform can isolate and route a session, but it does not know which human owns it. AgentCore explicitly states that it does not enforce session-to-user mappings, so the application backend must maintain the relationship between users and their session IDs and set per-user session limits. This is why the Asana incident is relevant to enterprise architecture.

Isolation was maintained while the user-to-session binding broke.

In that case, isolation was maintained while the user-to-session binding broke. The platform solves isolation and lifecycle, and it hands back identity mapping, authorization, and tenant context to the application. For an enterprise buyer, the key question is who owns that binding and how it is tested under concurrent, multi-tenant load.

How this differs from sticky sessions

A reasonable objection is that this appears to be sticky sessions combined with autoscaling on a microVM substrate. The distinction is worth drawing out. Traditional sticky sessions are usually performance optimizations, whereas in an agent runtime, session binding becomes a correctness and security requirement. Traditional load balancers route traffic to existing backends and do not own the lifecycle of the execution environment.

The new control plane provisions an environment when it first sees a session key, routes work to it, and tears it down after idle or lifetime limits. The primitive underneath is genuinely old, and its age is what makes the move legible. AWS open-sourced Firecracker at re:Invent in 2018 to pair VM-grade isolation with

powers Lambda and Fargate across trillions of executions a month. This is how cloud architecture tends to evolve, with existing primitives recombined around a new unit of abstraction.

The agent session is becoming the operational unit of enterprise AI infrastructure.

In summary, the important shift is not that cloud providers have found a better load balancer for agents. It is that the agent session is becoming the operational unit of enterprise AI infrastructure. Traditional load balancing routed independent requests to interchangeable workers. A session-aware runtime routes work to an isolated environment, preserves state across turns, and owns the lifecycle of the execution context. The dispatch layer did not disappear when agents arrived. It became session-aware, stateful, and lifecycle-aware, making it a control plane rather than a conventional load balancer.

Three practical questions will shape what comes next, and enterprises will need to evaluate each. The economics of per-session billing will decide how costs scale with concurrency. The open-source effort to rebuild this routing model on Kubernetes will determine its portability. The argument between microVMs and lighter isolates will decide the trade-off between isolation and density. For enterprises building agents, a runtime that owns isolation, routing, and lifecycle turns background plumbing into a deliberate architectural choice. It gives platform, security, and operations teams a foundation they can standardize on rather than rebuild for every workload.

TNS



Janakiram MSV (Jani) is a practicing architect, research analyst, and advisor to Silicon Valley startups. He focuses on the convergence of modern infrastructure powered by cloud-native technology and machine intelligence driven by generative AI. Before becoming an entrepreneur, he spent...

[Read more from Janakiram MSV →](#)

TRENDING STORIES

1. GitLab just surveyed 1,500 developers. Here's why it matters for your codebase.
2. AWS, Microsoft, and Google agree the session is the new unit of compute. They disagree on how to isolate it.
3. Neoclouds, sovereign AI and Postgres: The new operating model for regulated enterprises
4. OpenAI wants to claim more of the AI stack with Jalapeño, its first custom chip
5. "A data lake of nuance for AI agents to swim in": AWS Context gets shipshape on reasoning

INSIGHTS FROM OUR SPONSORS



The Hidden Cost of Kubernetes: How Over-Provisioning is Draining Your Cloud Budget

8 June 2026

Your Kubernetes Cluster Has GPUs. Now the Real Platform Problems Begin.

4 May 2026

We Are Introducing Devtron OSS Plus

26 February 2026



at Strategies Are Effective for Optimizing CDN Architecture?

in Oregon

23 June 2026

What Does Good CDN Reporting Look Like?

18 June 2026

ClickHouse

ClickHouse achieves AWS Cloud Operations Competency

26 June 2026

How I hunt for vulnerabilities with AI

26 June 2026

How Vibe.co handles billions of ad impressions with ClickHouse Cloud

25 June 2026

TNS DAILY NEWSLETTER

Receive a free roundup of the
most recent TNS articles in
your inbox each day.

svo1975pnv1982@gmail.com

SUBSCRIBE

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds



roadmap.sh

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)